

AWS Setup Request — Document Q&A

Create five AWS resources and send back one configuration file. Around 40 minutes.

What is being asked

A team wants to run a document question-answering tool on their own machines, against documents held in your AWS account. They have no console access, so everything that has to be created in AWS has to be created by you.

Nothing is being deployed into the account. There is no application to install, no server, no container, no VPC. The tool runs on the requester's laptop and calls AWS. What you create is storage and a search index; what you send back is the identifiers and the credentials to reach them.

#	Resource	Why	Required
1	Bedrock model access	Permission to call the model at all	Yes
2	S3 bucket — documents	Where the documents are stored	Yes
3	Bedrock Knowledge Base	Indexes and searches those documents	Yes
4	Bedrock Guardrail	PII masking and grounding checks	Recommended
5	S3 bucket — sessions	Keeps conversation history	Optional
6	Access for the requester	SSO or an access key	Yes

Cost. Storage is cents per month. There is nothing that bills while idle — no instance, no endpoint, no provisioned capacity. Charges appear per question asked, as Bedrock tokens, and per document indexed. A budget alert before handing over is sensible.

If you would rather not click through the console: the requester can supply a CloudFormation template that creates items 2–5 with the settings already correct. Ask them for it. The rest of this document is the manual route.

STEP 1 Enable Bedrock model access

This is the step that fails silently. If it is skipped, everything else still succeeds — resources create cleanly, nothing errors — and then every question fails with `AccessDeniedException`. It looks like a broken application and is not one.

Console → **Amazon Bedrock** → **Model access**, in the region you have chosen:

1. Enable a **Claude** chat model. Enable more than one if available — it turns a later change of mind into a config edit rather than another ticket.
2. Enable **Titan Text Embeddings V2** (`amazon.titan-embed-text-v2`). Nothing becomes searchable without it.
3. Wait for status *Access granted*.

Confirm — both commands must return at least one line:

```
aws bedrock list-inference-profiles --region <region> \
  --query "inferenceProfileSummaries[?contains(inferenceProfileId,'anthropic')].inferenceProfileId"

aws bedrock list-foundation-models --region <region> \
  --query "modelSummaries[?contains(modelId,'titan-embed-text-v2')].modelId"
```

STEP 2 **S3 bucket for documents**

Create a bucket in the same region. Defaults are fine — block public access on, encryption on.

Setting	Value
Region	Same as everything else
Versioning	Enabled — a bad load can then be undone
Public access	Blocked

Documents must live under a `documents/` prefix inside the bucket. The Knowledge Base in Step 3 is configured to look only there. Anything uploaded elsewhere in the bucket is silently never indexed.

STEP 3 Bedrock Knowledge Base

Console → **Amazon Bedrock** → **Knowledge Bases** → **Create**. This is the longest step and the one where a wrong setting is expensive: several of these cannot be changed afterwards without re-indexing every document.

Setting	Value	Note
Data source type	Amazon S3	
S3 URI	s3://<documents-bucket>/documents/	Include the prefix
Embeddings model	amazon.titan-embed-text-v2	Cannot change later
Vector dimensions	1024	Cannot change later
Vector store	Amazon S3 Vectors	Cheapest at rest; console can create it for you
Distance metric	cosine	
Chunking strategy	Hierarchical	Not fixed-size — see below
Parent chunk size	1500 tokens	
Child chunk size	300 tokens	
Overlap	60 tokens	

Why hierarchical matters. Small chunks are matched so the search is precise; the larger passage around the match is what gets read. Set to fixed-size instead and answers start losing the context around the thing they found — a table row returned without its header, for example. It costs a little more storage and is worth it.

After creating it, note the **Knowledge Base ID** — a short string like ABCD1234EF . Leave the data source empty for now; the requester loads the documents themselves.

STEP 4 Bedrock Guardrail — recommended

Console → **Bedrock** → **Guardrails** → **Create**. Skipping this is a valid decision, and it means answers are returned with no PII masking and no check that they are supported by the documents.

Section	Setting
Content filters	Hate, Insults, Sexual, Violence, Misconduct — strength High on input and output
Prompt attacks	Strength High on input. Output must be left <i>None</i> — Bedrock rejects a strength there
Sensitive information	Email, Phone, Name, Address, Credit/Debit card — action Mask , not Block
Contextual grounding	Grounding 0.75 , Relevance 0.5

Mask rather than Block is deliberate: an answer with a redacted phone number is still useful, a blocked answer is not.

Publish a version after creating it. Note the **Guardrail ID** and the **version number** — both are needed.

STEP 5 **S3 bucket for sessions** — optional

A second bucket, same region, defaults fine. It holds conversation history so a follow-up question remembers the previous one. Without it the tool still works, but every question starts a fresh conversation.

A lifecycle rule expiring objects after 30 days is sensible — this is transient state, not records.

STEP 6 Access for the requester

Choose whichever fits your policy.

Option A — SSO or an assumable role (preferred)

Grant the requester's identity the policy below and send them the SSO start URL. Nothing secret travels in the config file. They will run `aws configure sso` themselves.

Option B — an access key

Create an IAM user with the policy below, generate an access key, and include it in the config file. **The file then contains a live credential** — send it through a password manager or vault, not email or chat.

The policy — either way

This grants only what the tool does: call the model, search the knowledge base, load documents. **It cannot create or delete anything.** Replace the four placeholders.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "CallTheModel",
      "Effect": "Allow",
      "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
      "Resource": [
        "arn:aws:bedrock:REGION::foundation-model/*",
        "arn:aws:bedrock:REGION:ACCOUNT_ID:inference-profile/*",
        "arn:aws:bedrock:*:ACCOUNT_ID:inference-profile/*"
      ]
    },
    {
      "Sid": "SearchAndIndex",
      "Effect": "Allow",
      "Action": [
        "bedrock:Retrieve",
        "bedrock:ListDataSources",
        "bedrock:StartIngestionJob",
        "bedrock:GetIngestionJob"
      ],
      "Resource": "arn:aws:bedrock:REGION:ACCOUNT_ID:knowledge-base/KB_ID"
    },
    {
      "Sid": "ApplyTheGuardrail",
      "Effect": "Allow",
      "Action": "bedrock:ApplyGuardrail",
      "Resource": "arn:aws:bedrock:REGION:ACCOUNT_ID:guardrail/GUARDRAIL_ID"
    },
    {
      "Sid": "LoadDocuments",
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::DOCUMENTS_BUCKET",
        "arn:aws:s3:::DOCUMENTS_BUCKET/*"
      ]
    },
    {
      "Sid": "ConversationState",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::SESSION_BUCKET",
        "arn:aws:s3:::SESSION_BUCKET/*"
      ]
    }
  ]
}

```

The two `inference-profile` lines are not a duplicate. Claude is reached through cross-region profiles, so a call made in one region may be served from another. **If your policy requires data to stay in one region, say so when you send this back** — the requester will pin a single-region profile instead, and that is a change they must make.

STEP 7 Fill in and send

Copy this, fill in the values, send it back. Delete whichever credential block does not apply.

```
# --- Region -----
AWS_REGION=

# --- Credentials: Option A (SSO) -----
# Leave the keys below out entirely. Send the start URL:
#   SSO start URL:
#   Account id:
#   Role / permission set name:

# --- Credentials: Option B (access key) -----
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=

# --- Resources -----
# Step 3 – the short id, e.g. ABCD1234EF
KNOWLEDGE_BASE_ID=

# Step 2 – bucket name only, no s3:// and no prefix
DOCUMENTS_BUCKET=

# Step 5 – leave blank if it was skipped
SESSION_BUCKET=

# Step 4 – leave both blank if it was skipped
GUARDRAIL_ID=
GUARDRAIL_VERSION=

# Step 1 – the chat model that was enabled, e.g.
#   global.anthropic.claude-sonnet-4-6
MODEL_ID=
```

Leave a value blank rather than guessing. A blank means "not provided" and the tool says so clearly. A wrong identifier fails much later, with an error that points at the wrong thing.

Anything else worth telling them

- Whether data-residency rules apply (see the note in Step 6).
- The *Knowledge bases per account* quota, if you know it — it is the ceiling on how many separate document sets they can have.
- Who to contact if a permission turns out to be missing. This is the most likely thing to need a second pass.

If something looks wrong

Symptom	Cause	Fix
Knowledge Base will not create	Embedding model not enabled	Step 1, same region
Knowledge Base creates but indexes nothing	Documents not under documents/	Move them, re-sync

Requester reports AccessDeniedException	Model access off, or off in a different region	Step 1
Requester reports a throttling error	Bedrock quota too low	Request an increase for that model
Guardrail rejects an output strength	Prompt attacks has no output setting	Leave output as <i>None</i>

Sent alongside this: nothing else is required. The requester installs and runs the tool themselves and does not need access to create, modify or delete any AWS resource.